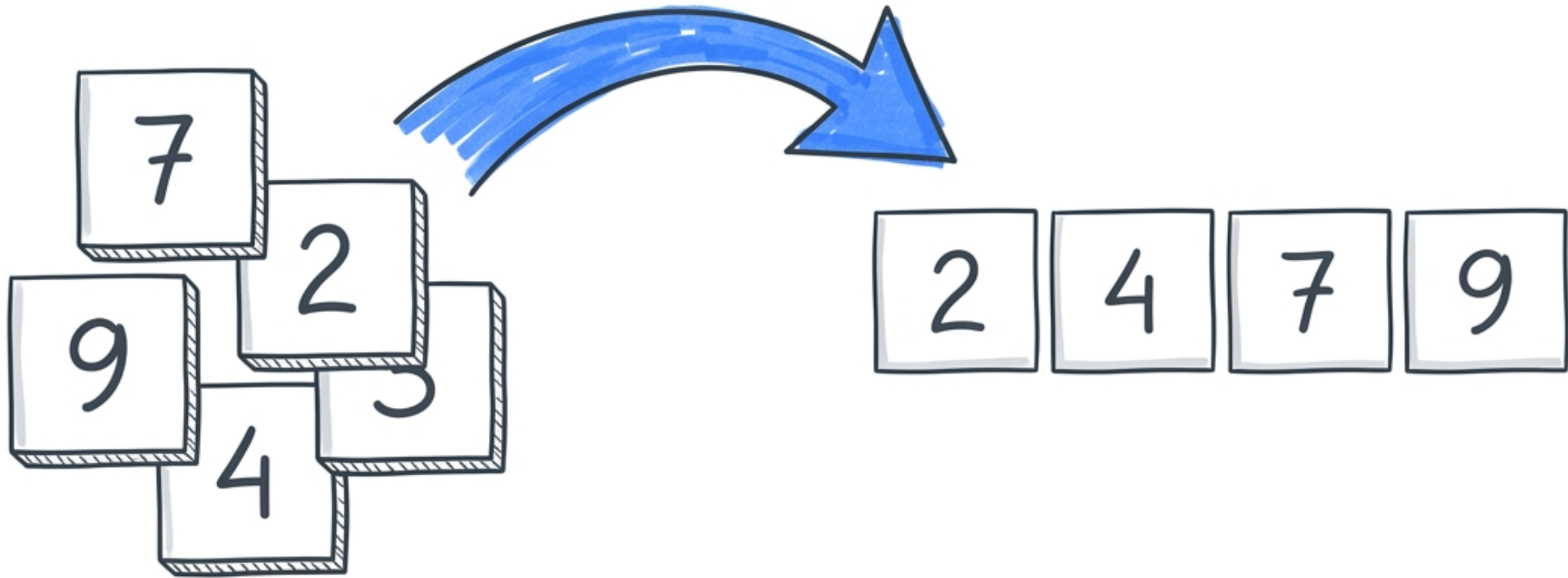


Métodos de Ordenamiento

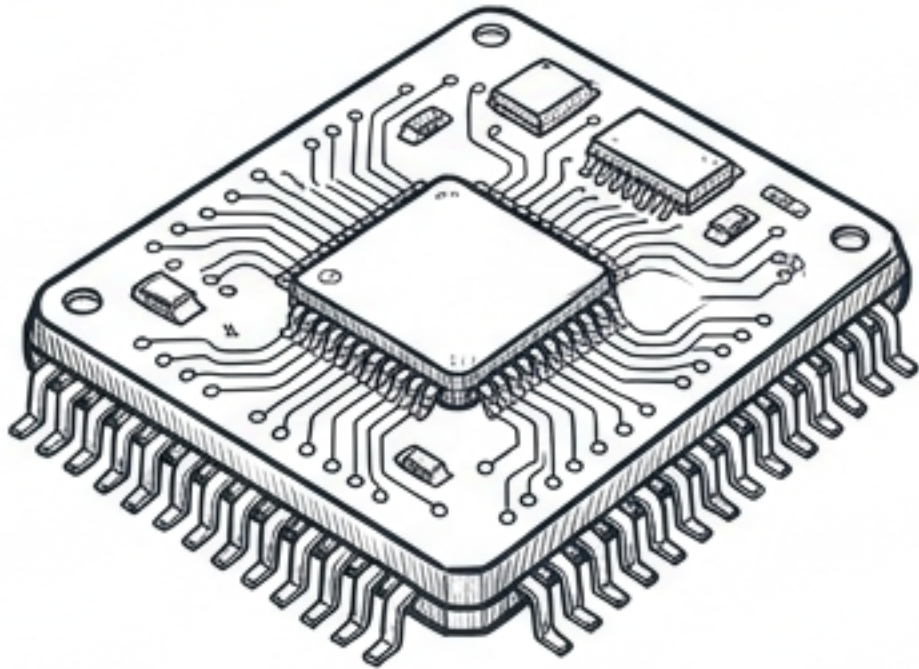
De la lógica básica a la eficiencia algorítmica.



¿Qué es la Ordenación?

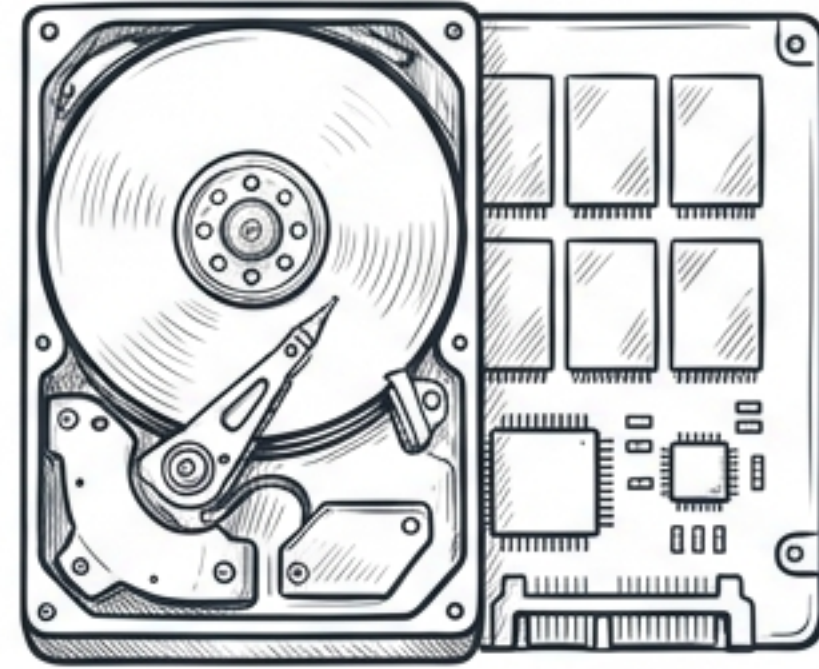
Disponer un conjunto de datos en un orden específico respecto a un campo clave (**k**).

Ordenación Interna



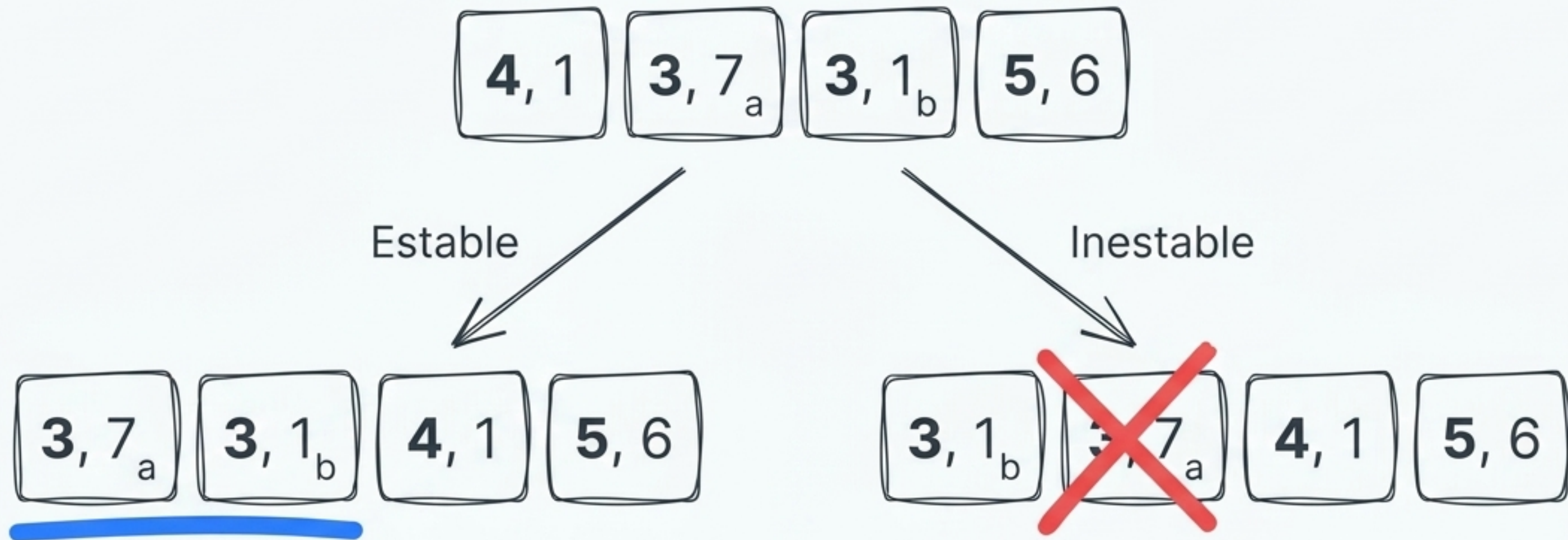
Los datos a ordenar caben completamente en la memoria principal (arrays, listas).

Ordenación Externa



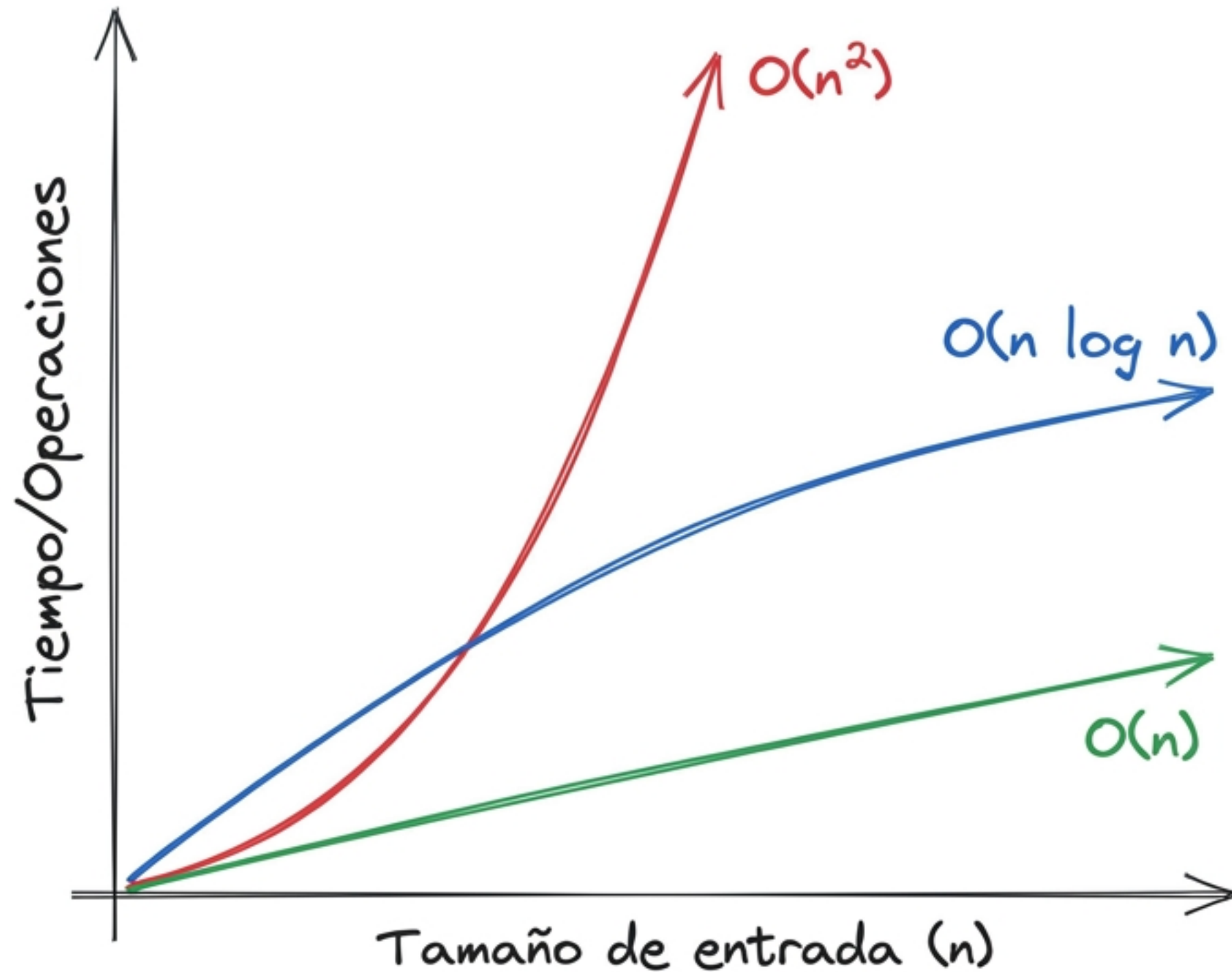
Volúmenes masivos de datos; requieren almacenamiento externo permanente (archivos, bases de datos).

Propiedades Clave: La Estabilidad



Un algoritmo estable mantiene el orden relativo original de los elementos con claves iguales.

Evaluando la Eficiencia: Notación Big O



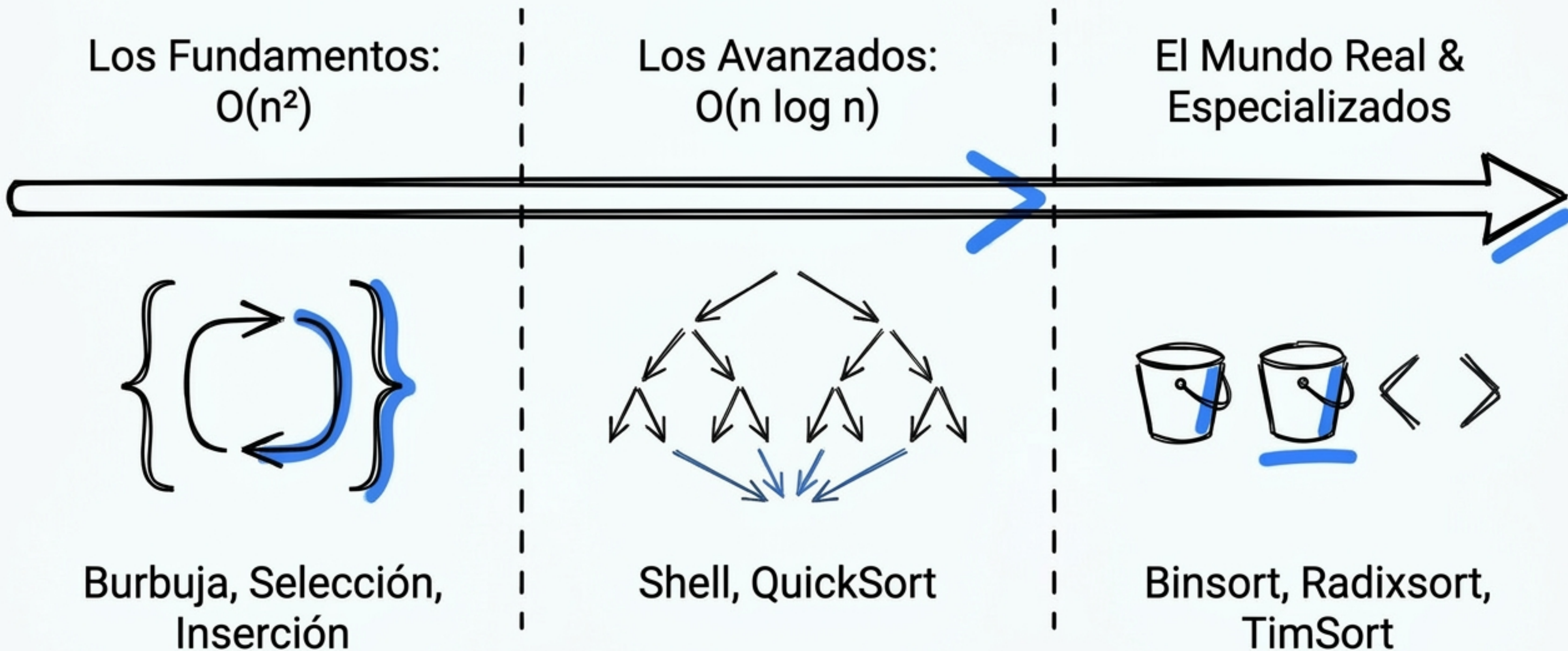
Eficiencia: Se mide aislando operaciones clave (comparaciones e intercambios).

Mejor Caso: El array ya está ordenado.

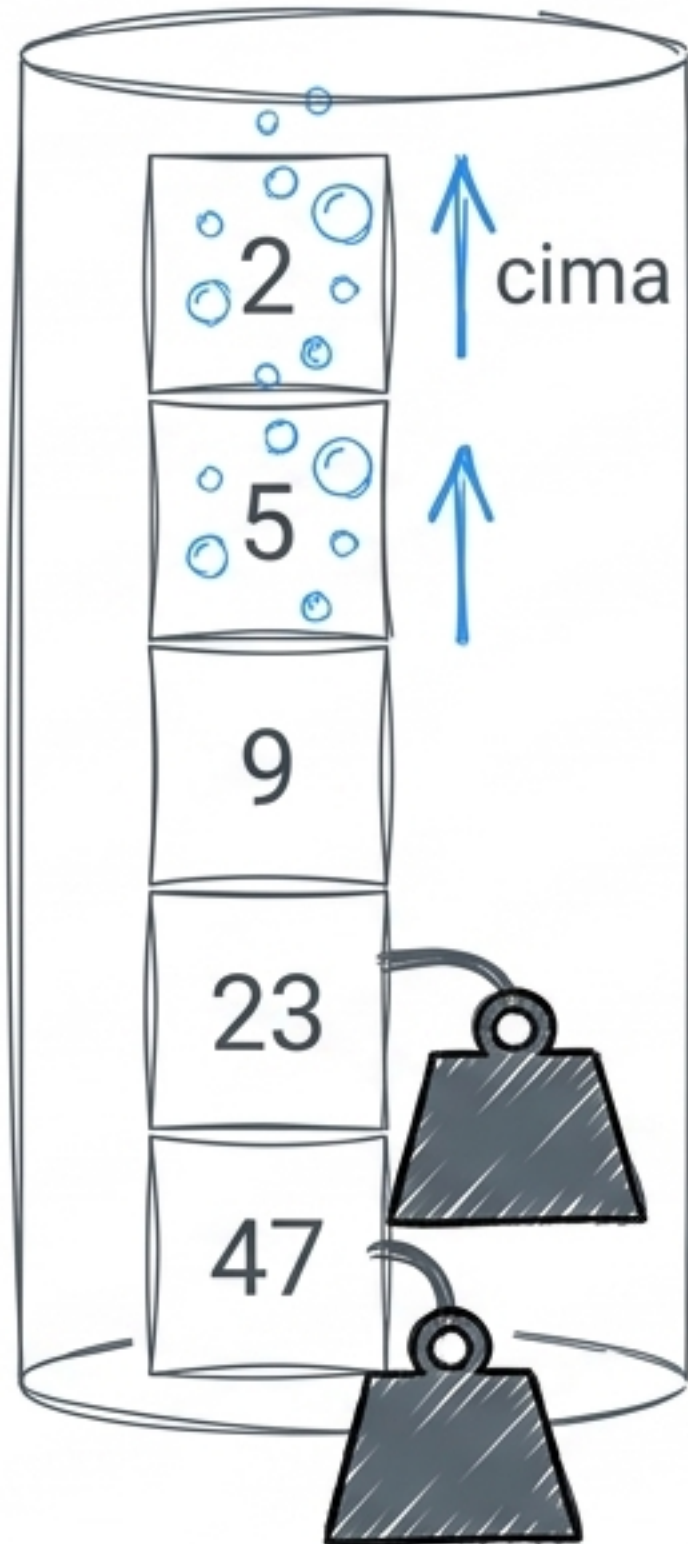
Peor Caso: El array está en orden inverso o la selección del pivote es pobre.

Caso Promedio: Comportamiento general con datos aleatorios.

La Evolución de los Algoritmos



Ordenación por Burbuja (Bubble Sort)



Compara **parejas sucesivas** de **elementos**.

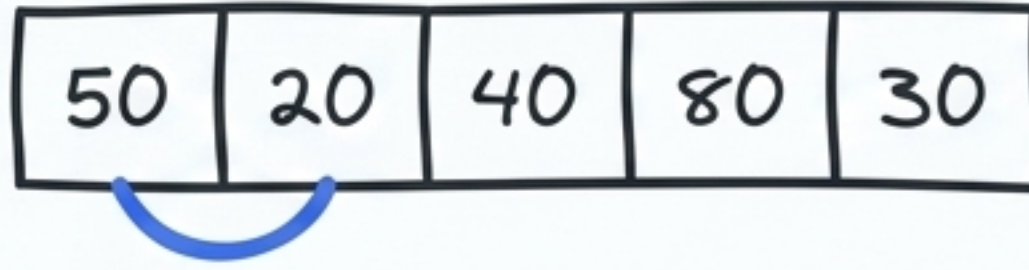
Si están en orden **decreciente**, se **intercambian**.

Los valores pequeños '**burbujean**' hacia la **cima**; los **mayores** se **hunden**.

Optimización: Usar una variable **interruptor** para detener el proceso si no hubo intercambios en una pasada.

Burbuja en Acción (Pasada 0)

Paso 1:



Intercambio 50 y 20

Paso 2:



Intercambio 50 y 40

Paso 3:



Ordenados. No hay intercambio.

Paso 4:



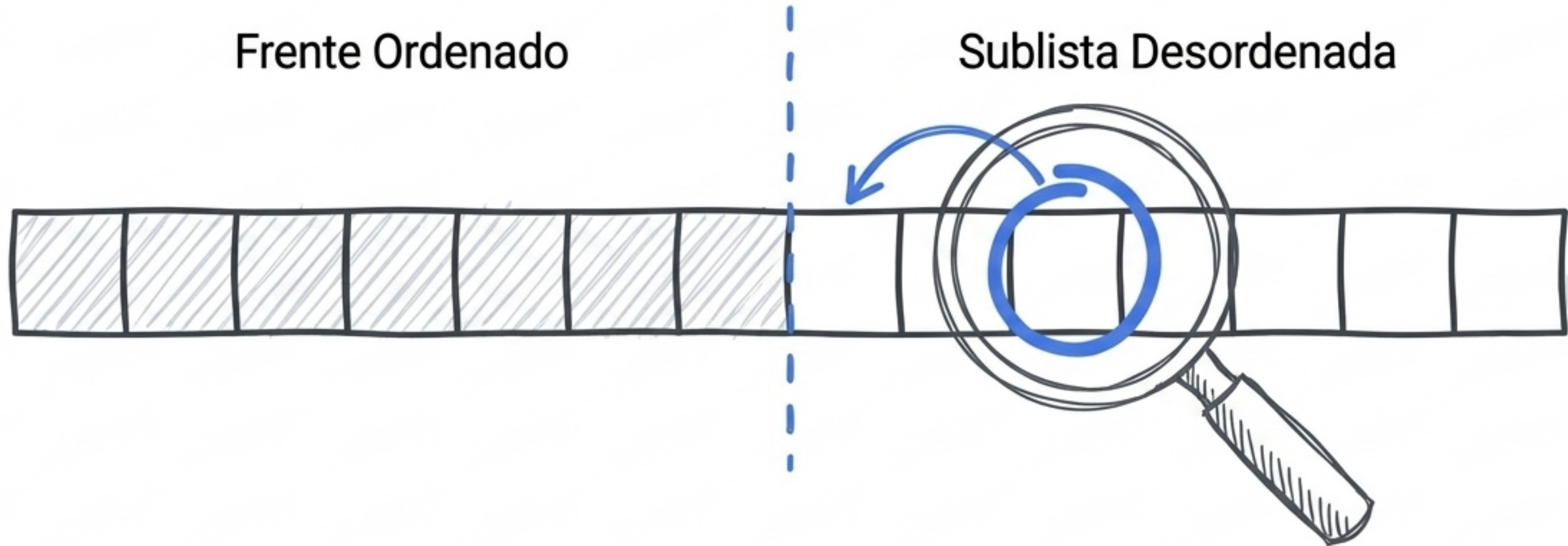
Intercambio 80 y 30

Resultado:



El elemento mayor está en su posición final

Ordenación por Selección



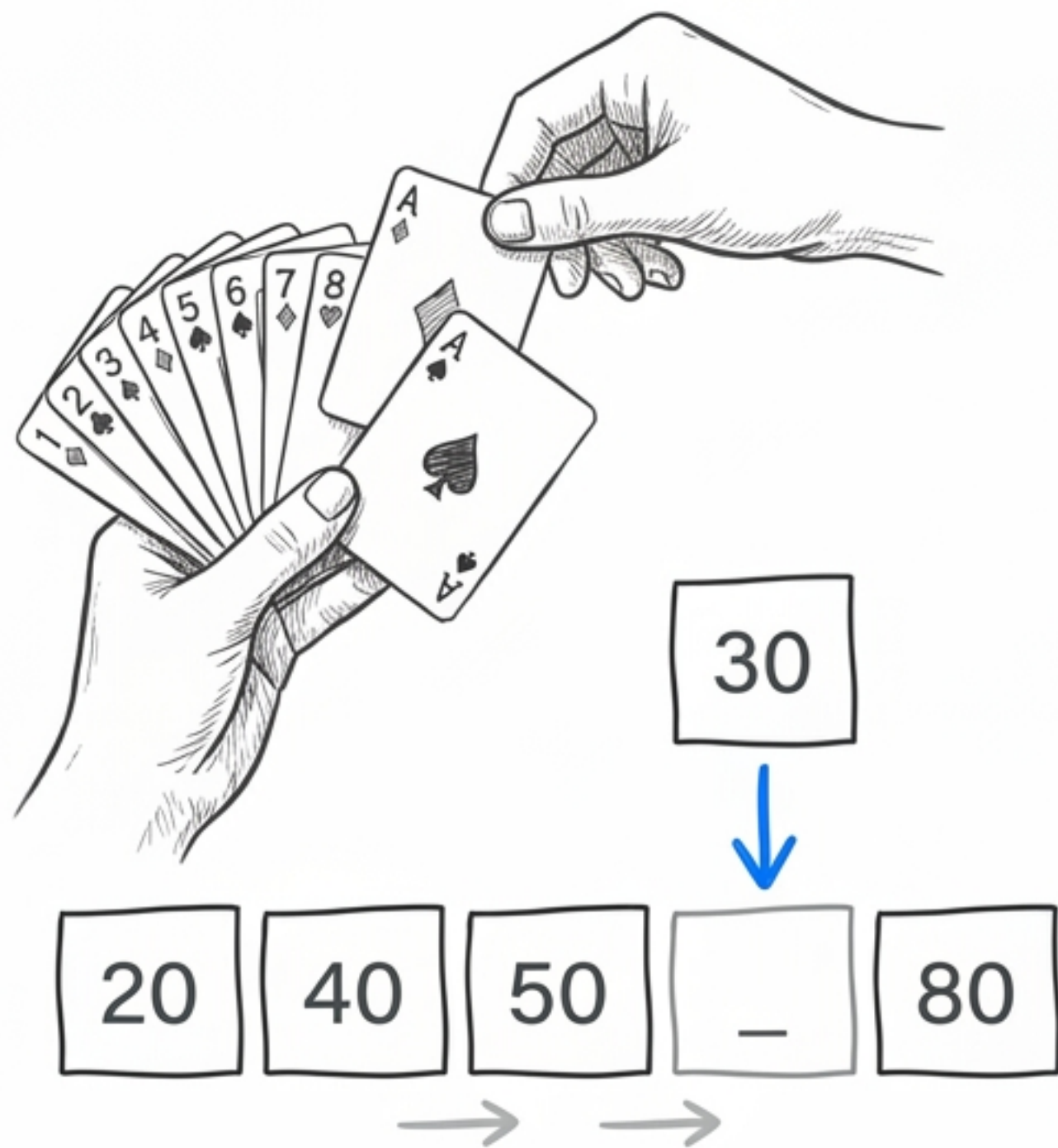
- Busca el elemento más pequeño de la sublista desordenada.
- Lo intercambia con el primer elemento de dicha sublista.
- Requiere un número fijo de comparaciones, independientemente del orden inicial.

Selección en Acción



El límite del área ordenada (línea azul) avanza un paso por iteración.

Ordenación por Inserción



Asume que el primer elemento está ordenado.

Toma el siguiente elemento y explora hacia atrás.

Desplaza los elementos mayores hacia la derecha para hacer espacio.

Inserta el elemento en su posición correcta.

Inserción en Acción

Paso 1:



Comienzo con 50

Paso 2:



Paso 3:



Paso 4:



No requiere desplazamiento

Paso 5:



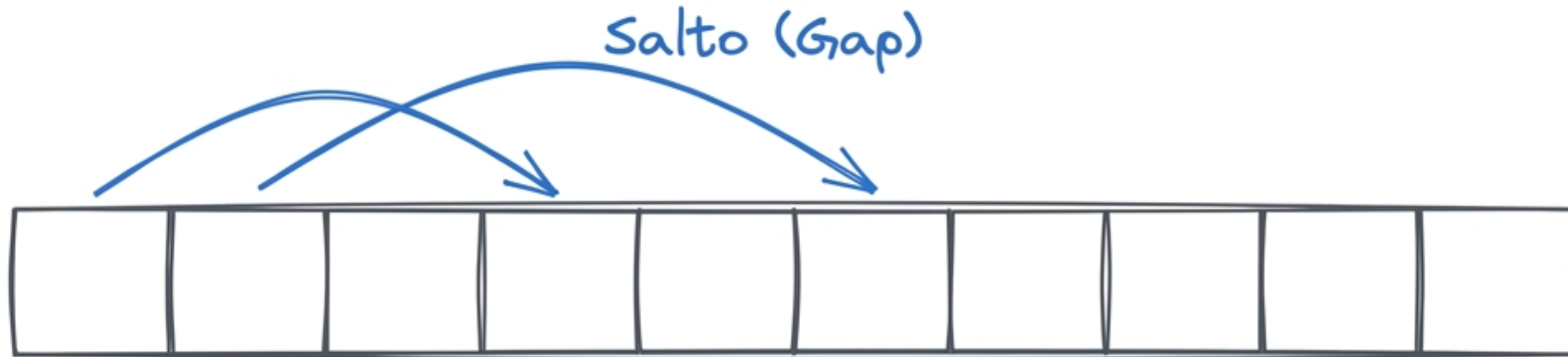
Muy eficiente para listas pequeñas o 'casi ordenadas'.

Resumen: El Muro de la Complejidad Cuadrática

Método	Características	Veredicto
Burbuja $O(n^2)$	Intercambios constantes.	Educativo, pero ineficiente en la práctica.
Selección $O(n^2)$	Intercambios fijos $(n-1)$.	Predecible, pero lento con datos grandes.
Inserción $O(n^2)$ Mejor: $O(n)$	Desplazamientos locales.	El mejor de los métodos directos; usado internamente por algoritmos avanzados.

A medida que la cantidad de datos (n) crece, el costo computacional de $O(n^2)$ se vuelve prohibitivo. Necesitamos romper la lista.

Ordenación Shell: Inserción con Saltos



Mejora de la inserción directa inventada por D.L. Shell.

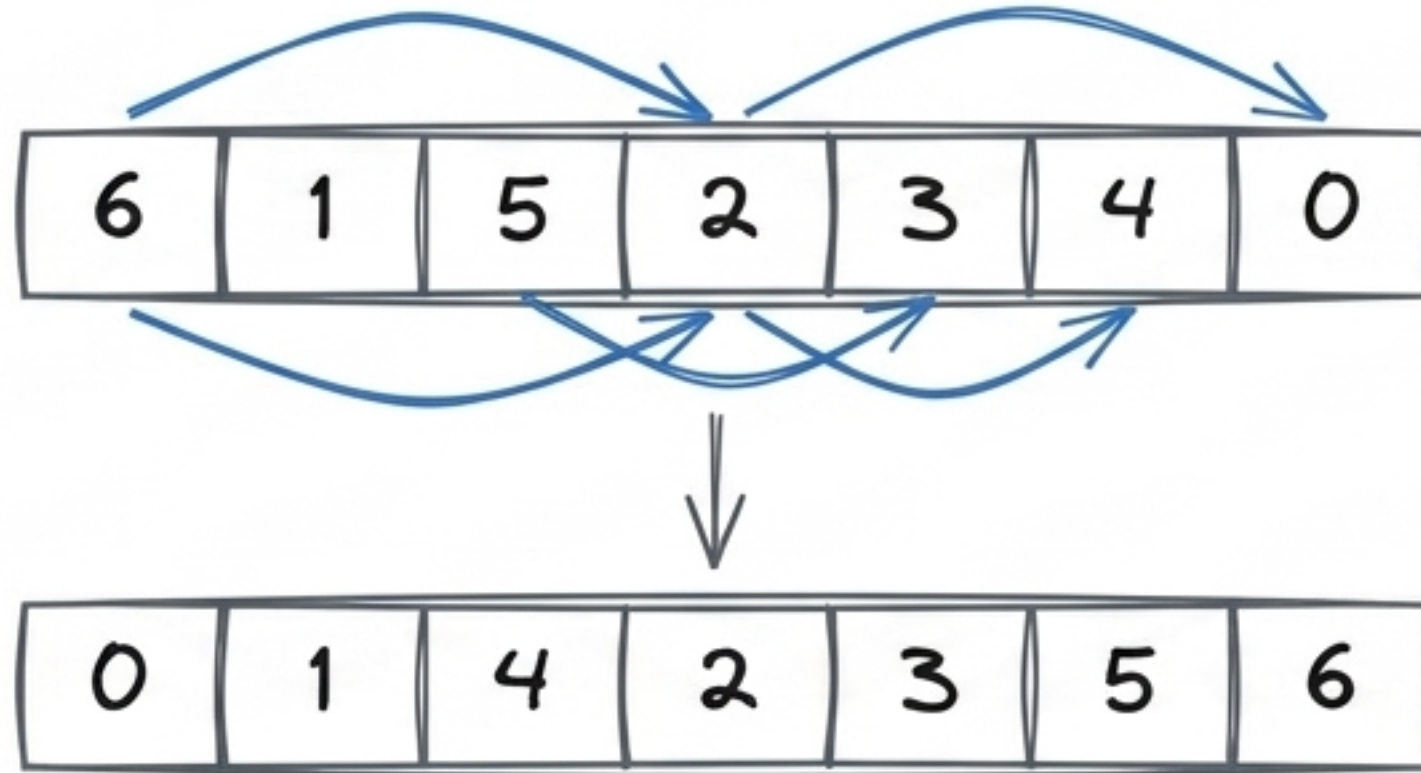
En lugar de mover elementos paso a paso, realiza comparaciones con 'saltos' (gaps).

El intervalo se reduce progresivamente (ej. $n/2$, $n/4$... 1).

Permite que los elementos se acerquen a su posición final mucho más rápido.

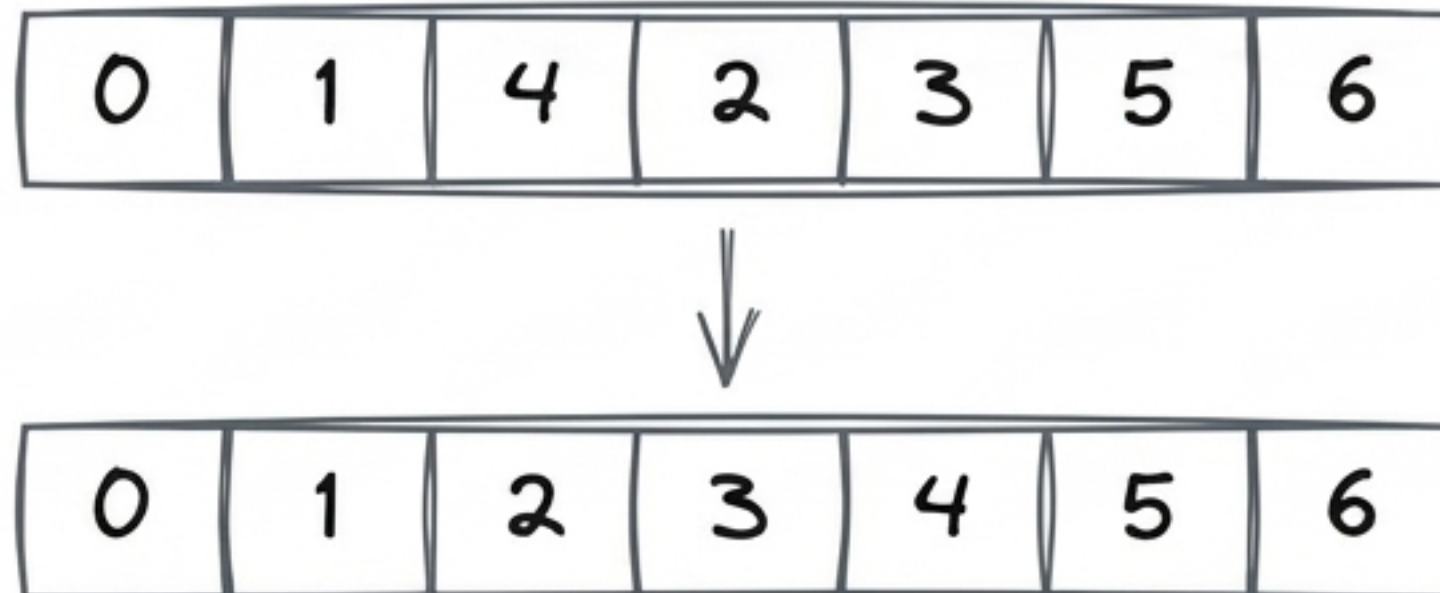
Shell Sort en Acción

Salto 3 ($n/2$):



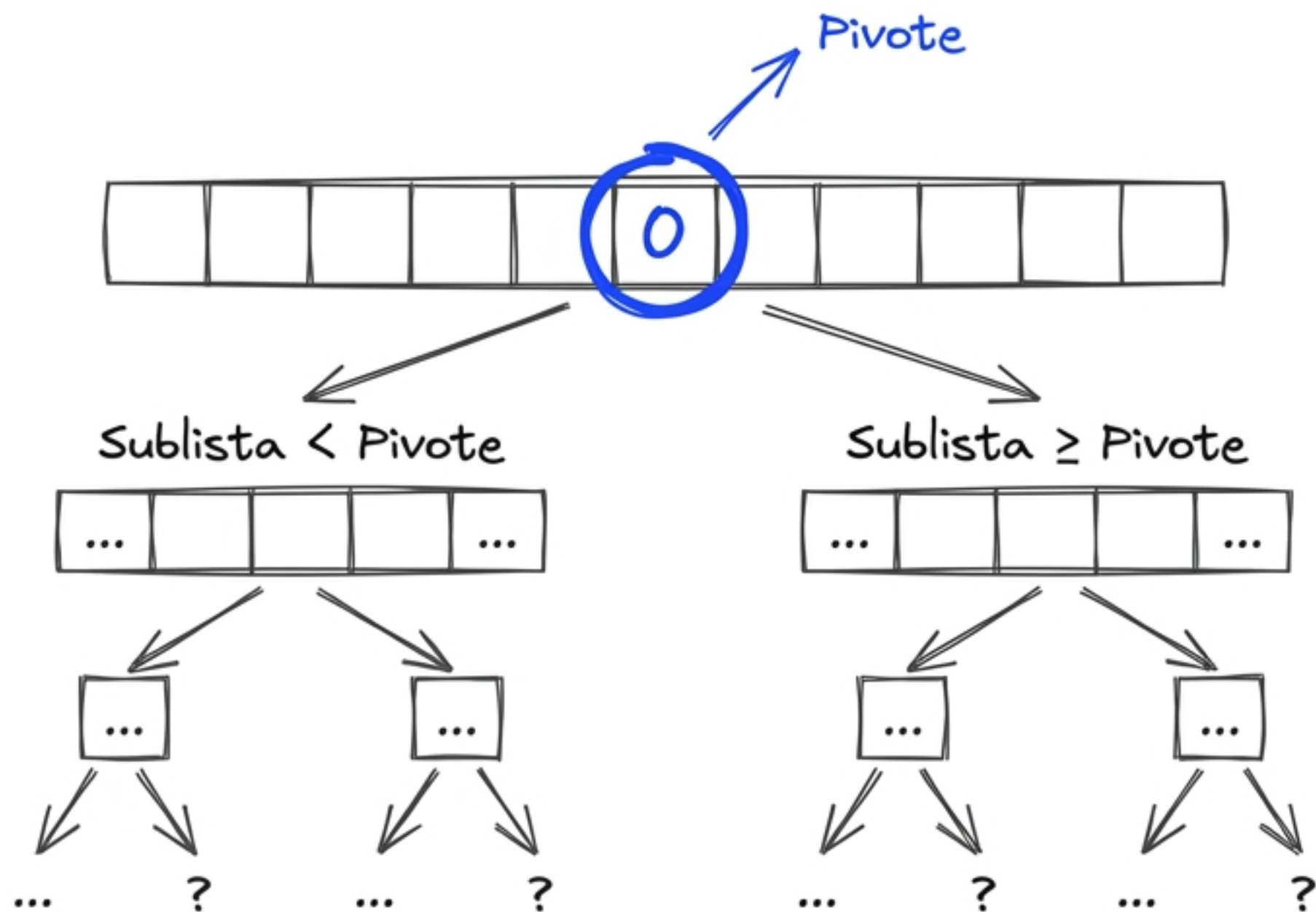
Sublistas separadas por 3 posiciones se ordenan mediante inserción.

Salto 1:



El salto se reduce a 1 (Inserción normal). Como la lista ya está 'casi ordenada', los desplazamientos son mínimos.

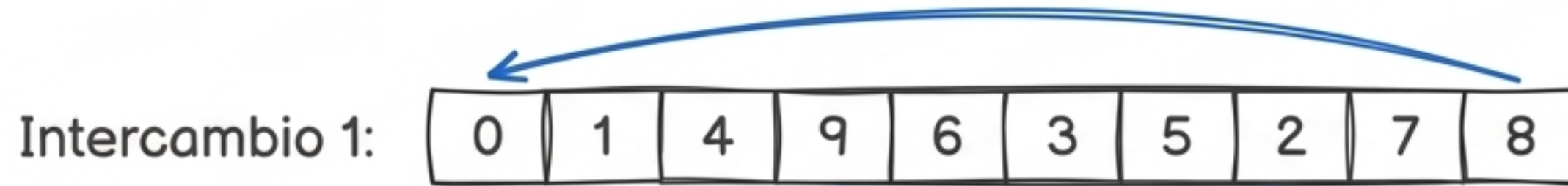
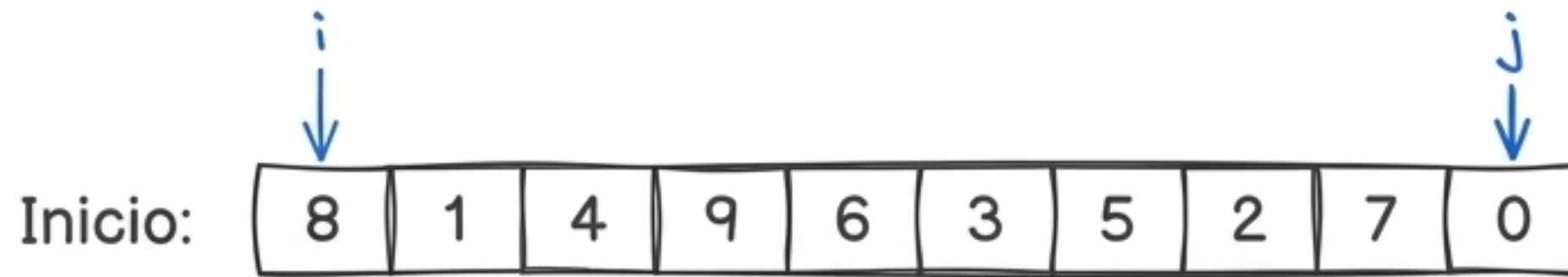
QuickSort: Divide y Vencerás



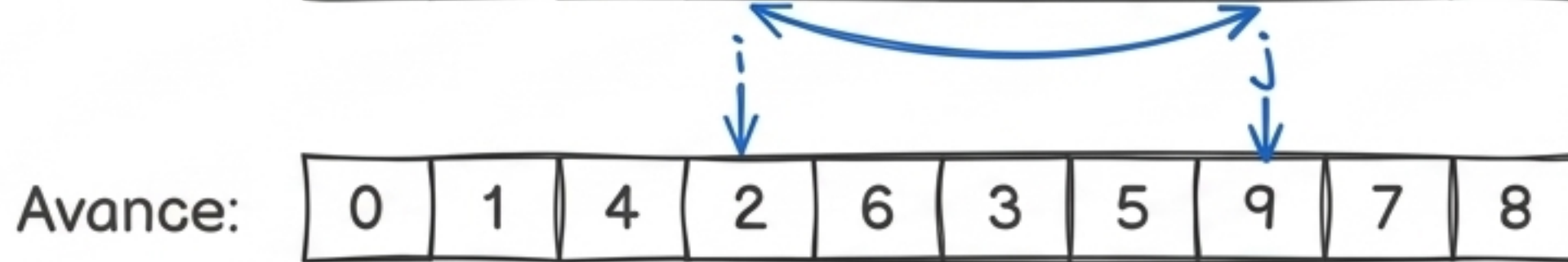
- ✦ Estrategia: Elegir un elemento como "Pivote".
- ✦ Partición: Reorganizar la lista para que los menores al pivote queden a la izquierda, y los mayores a la derecha.
- ✦ Recursividad: Aplicar el mismo proceso independientemente a ambas sublistas.

QuickSort: La Partición

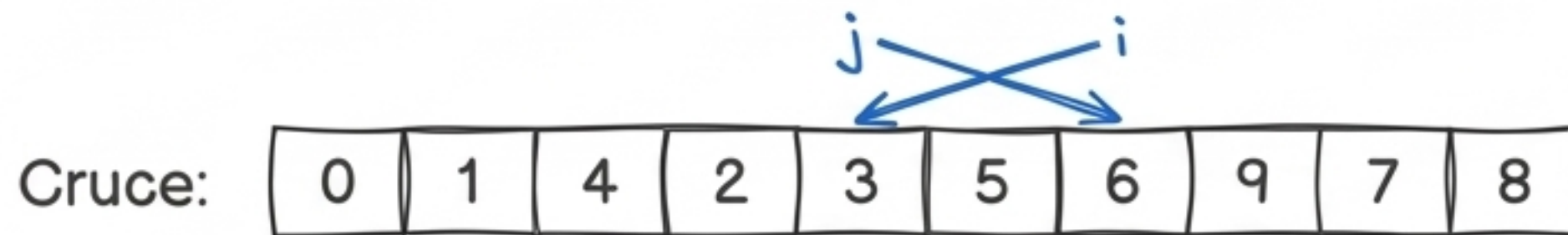
Array fuente: [8, 1, 4, 9, 6, 3, 3, 2, 7, 0] | Pivote = 6



8 es mayor a 6, 0 es menor a 6.
Se intercambian.



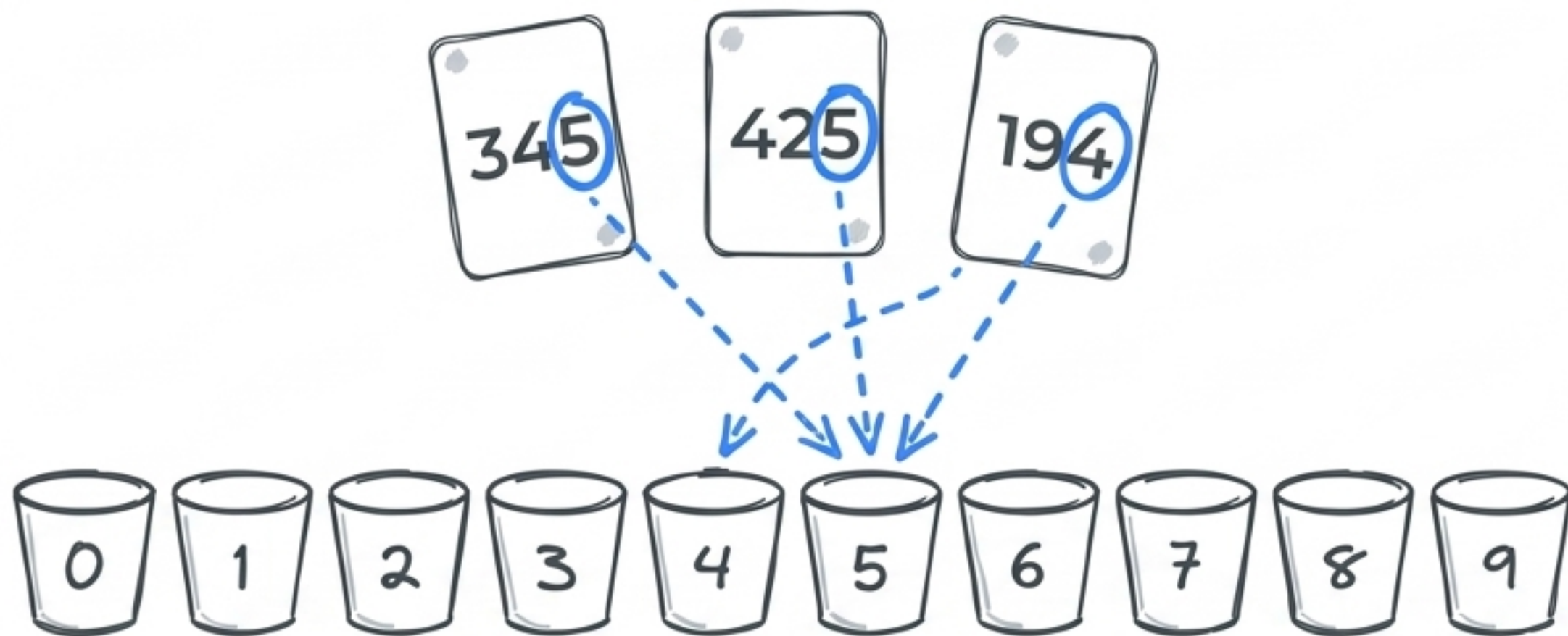
Puntero 'i' avanza y se detiene en '9' (mayor que 6).
Puntero 'j' retrocede y se detiene en '2' (menor que 6). Se intercambian.



Punteros se cruzan. Partición completada.
El **pivote 6** queda en su posición correcta (entre 5 y 9).

La magia ocurre in situ (en la misma memoria),
usando punteros que convergen desde los extremos.

Métodos No Basados en Comparación (Binsort & Radixsort)



Aprovechan la estructura interna de las claves (como dígitos o caracteres).

Urnas (Buckets): Distribuye los elementos en colas.

Radixsort: Clasifica repetidamente dígito por dígito (desde el menos significativo al más significativo).

Logra tiempos menores a $O(n \log n)$ bajo condiciones específicas.

En el Mundo Real: Python

`lista.sort()`

Lista original

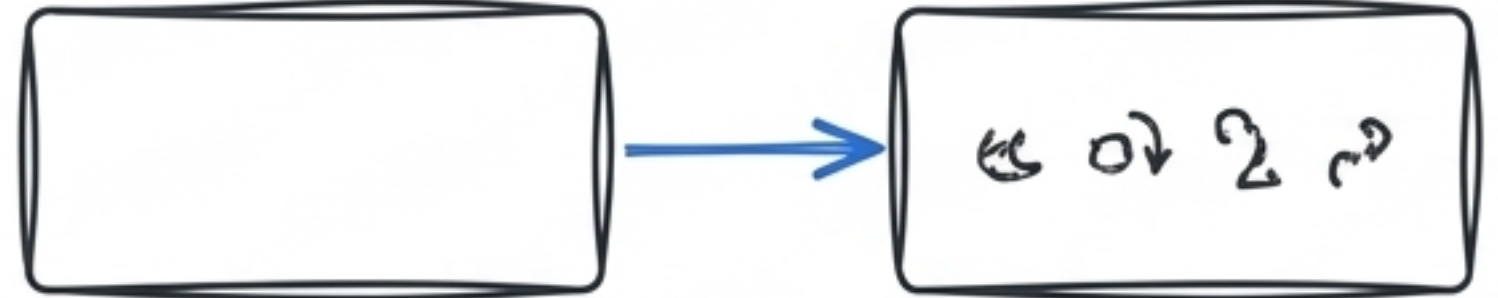


Ordena in situ. Modifica la lista original.
Devuelve None.

`sorted(lista)`

Lista original

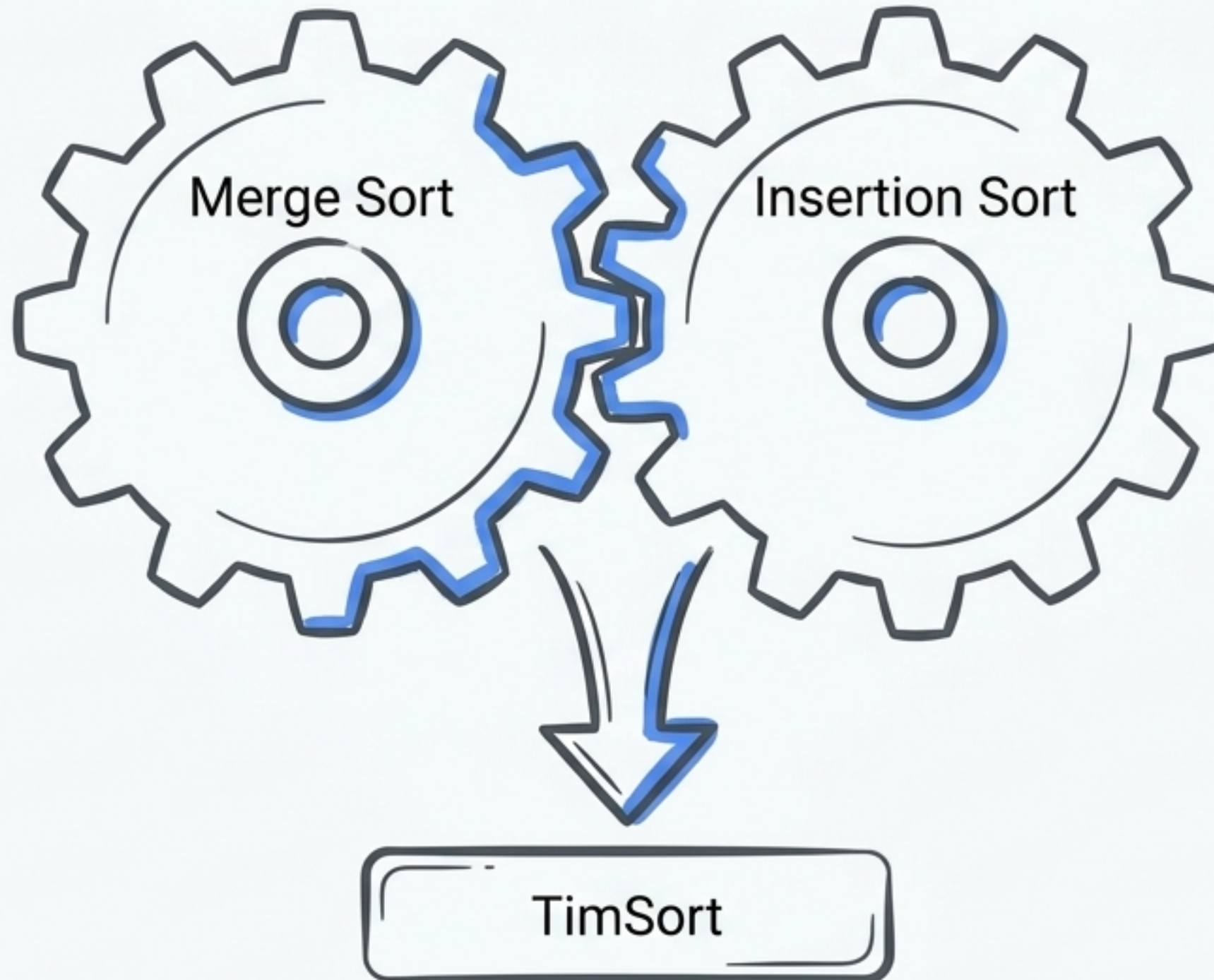
Nueva Lista Ordenada



Devuelve una nueva lista.
Acepta cualquier iterable.

Ambos métodos permiten utilizar parámetros 'key' y 'reverse' para ordenar por criterios personalizados.

TimSort: El Motor Híbrido Moderno



Algoritmo por defecto en Python, Java SE, Android y Rust.

Es Estable e Híbrido.

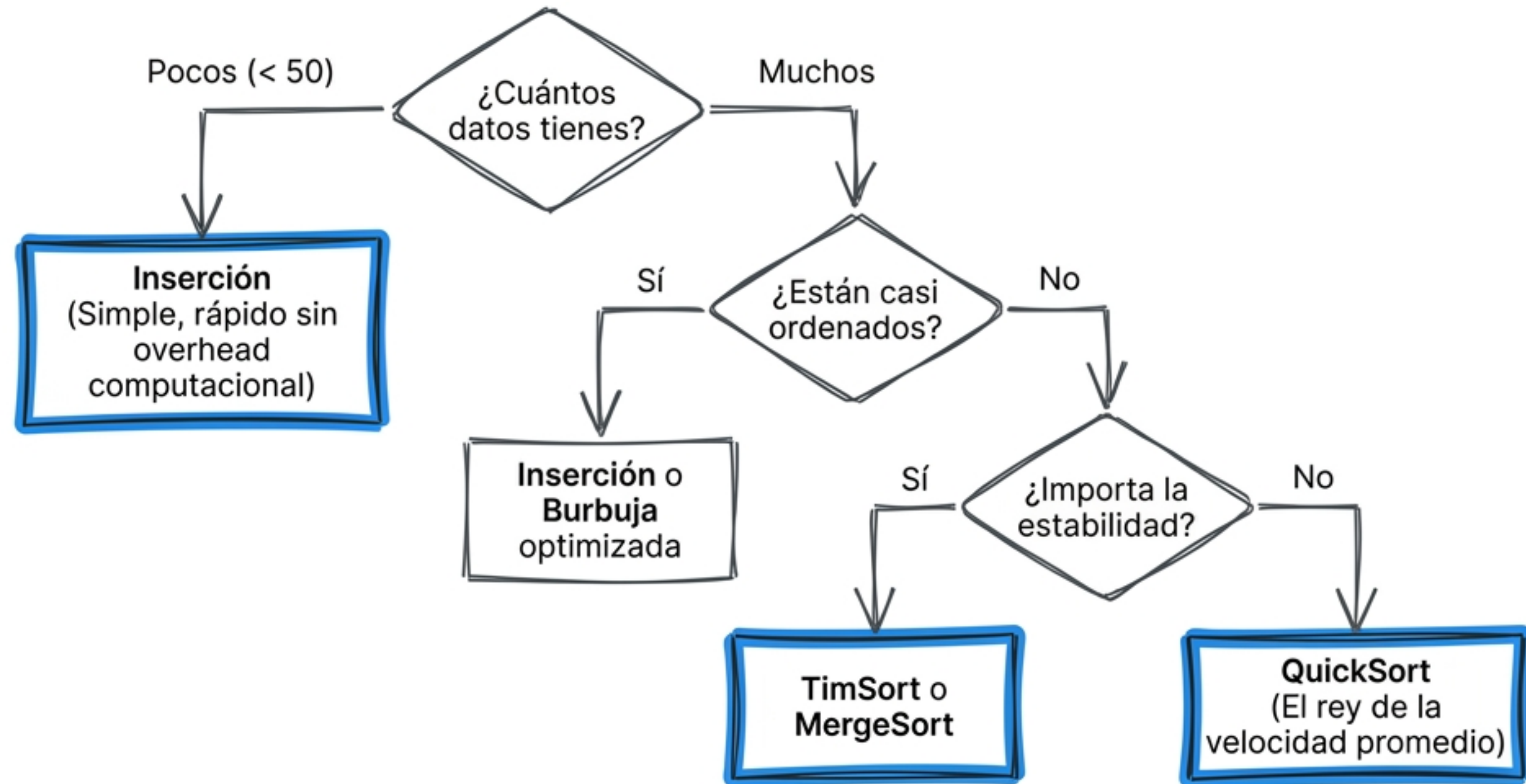
Aprovecha las secuencias que ya están parcialmente ordenadas en los datos del mundo real.

Cuando las sublistas son pequeñas (ej. menores a 64 elementos), utiliza Inserción por su inmensa rapidez a pequeña escala.

Matriz de Complejidad Computacional

Método	Mejor Caso	Caso Promedio	Peor Caso	Estable
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Shell	$O(n \log n)$	$O(n^{4/3})$	$O(n^2)$	No
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
TimSort	$O(n)$	$O(n \log n)$	$O(n \log n)$	Sí

¿Qué Algoritmo Elegir?



No existe un algoritmo perfecto; existe el algoritmo adecuado para los datos adecuados.